Autonomous Vehicle Challenge (AVC) Final Report

Student Name: Anzar Tatlik

Student ID: 300491855

Course Code: ENGR101
Lecturer: Arthur Roberts
Assignment: AVC Final Report
Date: 10 June 2025

Abstract

Autonomous navigation demands systems that can interpret sensor input and react dynamically in uncertain environments — a key capability in robotics and embedded systems. This project investigates the implementation of real-time, vision-based control using a modular robot that navigates a multi-phase obstacle course. The robot was engineered to perform network-based communication, camera-guided line tracking, intersection decision-making, and colour-based object handling. A lightweight, state-driven architecture was paired with a custom exponential PID controller to enhance stability and responsiveness. The system demonstrated robust performance in simulation and partial physical success under real-world conditions, particularly excelling in smooth line-following and quadrant transitions. Although the robot struggled with repeated sharp turns in the final test, the project validated key control strategies and reinforced the value of modular design, dynamic sensor processing, and principled logic under tight development constraints.

1. Introduction

This report presents the design and implementation of an autonomous robot developed for the AVC project in ENGR101. The focus lies on the robot's software architecture, real-time image processing, control systems, and quadrant-specific navigation logic.

Autonomous navigation requires real-time integration of sensing, decision-making, and actuation — a key challenge in engineering. This project mirrors real-world applications such as autonomous vehicles and robotic automation, serving as an applied introduction to embedded systems, control theory, and sensor-driven design.

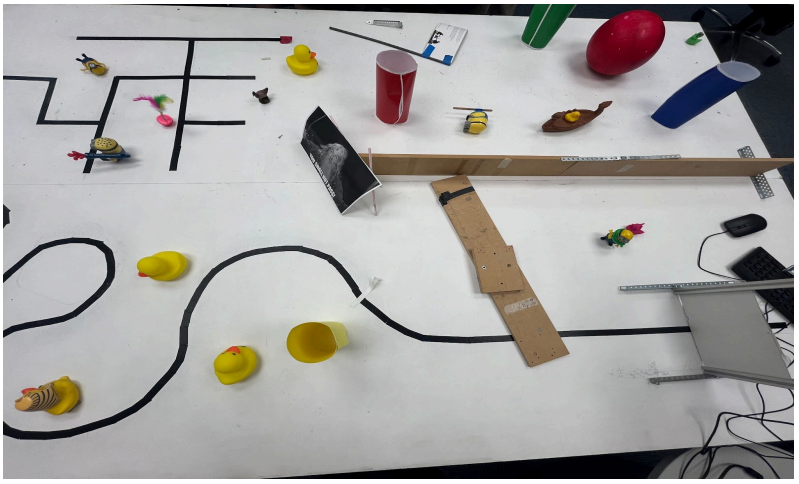
The primary aim was to develop a modular and intelligent robot capable of autonomously navigating all four quadrants of the AVC course using adaptive control strategies.

Objectives:

- Implement a dynamic line-following algorithm capable of handling curves and intersections with minimal deviation.
- Design a lightweight, state-driven control mechanism for turn decisions in quadrant 3.
- Integrate a nonlinear PID control strategy to ensure smooth and responsive motor output.
- Apply real-time colour detection to identify and respond to quadrant transitions based on object cues.

The project provided practical experience in real-time robotics, modular C++ development, and vision-based control systems — skills directly applicable to future work in embedded systems, autonomous vehicles, and control engineering.

2. Background



Autonomous navigation in robotics typically involves real-time perception, decision-making, and actuation. The AVC course simulates this pipeline by dividing the challenge into four distinct quadrants, each targeting a key robotic competency:

- **Quadrant 1** tests communication protocols by requiring the robot to exchange messages with a remote server to open a gate.
- **Quadrant 2** assesses vision-based line following, where the robot must track a curved black line using image processing techniques.
- **Quadrant 3** focuses on decision-making under uncertainty, involving sharp turns and T-junctions that require dynamic state awareness.
- **Quadrant 4** evaluates object detection and localization through the use of coloured targets, testing multi-spectral analysis and control feedback.

The line-following system relies on real-time grayscale image analysis, where a dynamically computed brightness threshold is used to identify black pixels. A weighted centroid is calculated from these pixels to estimate the error from the line's center. This visual error informs a non-linear PID controller, which adjusts the robot's steering based on shaped proportional and derivative gains.

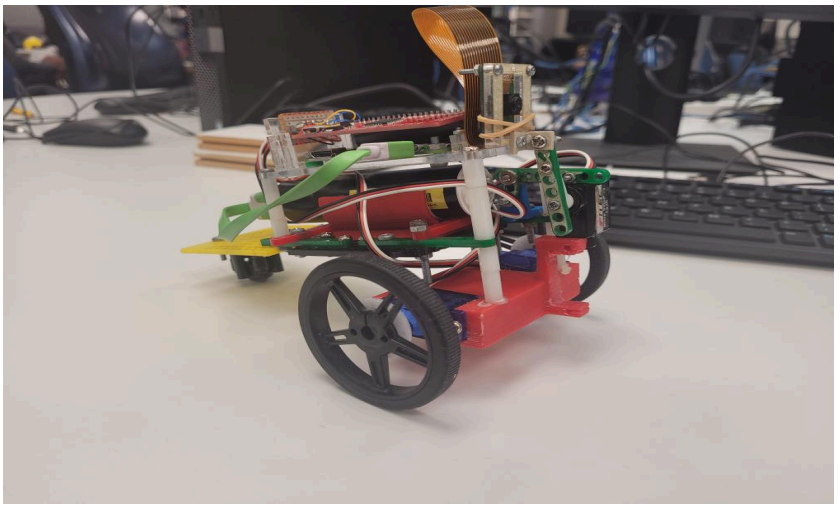
In contrast to conventional PID control, the system implements **exponential shaping**, allowing stronger corrections for large errors while dampening small oscillations. This technique provided a smoother response without requiring gain retuning across scenarios.

For Quadrant 3, the robot uses a simplified internal state logic instead of a full finite state machine (FSM). This includes local flags and cooldown timers to manage sharp turns and intersection decisions. Such a lightweight approach is advantageous in resource-constrained environments.

3. Method

3.1 Hardware Configuration

The robot platform consisted of a Raspberry Pi Zero W for processing, connected to a standard Raspberry Pi camera module mounted on a tilt servo (motor M5). Locomotion was handled by two drive motors (M1 and M3), each connected to the wheels in a differential drive configuration. Power was supplied via a portable USB power bank. All components were integrated on a custom chassis with appropriate gearing and cable management.



3.2 Software Architecture

The software system was written in C++ and divided into modular components:

- `AVC_ver3.cpp` contained the primary logic and quadrant-based finite state control.
- `AVC_utils.cpp/.h` provided helper functions for line detection, speed control, motor actuation, and image processing.
- A Makefile facilitated compilation and modular testing through custom test files such as `Q1_test.cpp`, `Q3_test.cpp`, etc.

3.3 Line Detection and Analysis

Image capture and processing are handled through the E101 library's core functions, including `take_picture()` and `get_pixel()`. These allow per-frame analysis at the pixel level, forming the basis for both line detection and colour recognition.

To detect black tape for line-following, the robot captured grayscale frames using the `take_picture()` function. Each frame was scanned along a horizontal row (row 90). The minimum and maximum brightness values of the row were used to calculate a dynamic threshold:

$$Threshold = \frac{min_brightness + max_brightnesst}{2}$$

Pixels darker than this threshold were classified as black. The horizontal centroid of these black pixels was computed using a weighted sum, allowing the robot to estimate the line's horizontal offset (error) from the image center. Separately, the entire frame was scanned for red pixel clusters, which served as visual cues for quadrant transitions — such as detecting a red flag to initiate entry into Q3.

3.4 PID Steering Control

The computed line error was passed into a custom PID-like controller with exponential gain shaping. Instead of fixed gains, both the proportional and derivative terms were scaled based on the magnitude of the error to create smoother low-speed corrections and stronger reactions to large deviations.

The gain shaping was defined as:

$$shaped = \left(min\left(\frac{|e|}{20}, 1.0\right)\right)^{1.2}$$

where:

$$KP = 0.55 + 0.22 \cdot shaped, \quad KD = 0.20 + 0.22 \cdot shaped$$

These gains were applied to calculate the control signal:

$$dv = KP \cdot e + KD \cdot \Delta e$$

Additional enhancements were implemented, including:

- **Sign flip damping** (temporary increase in KD after direction change)
- **Center fade zone** (ignore or reduce dv near zero error)
- **Boost factor** (for large errors)

A constant base speed (PWM = 48 $\pm$ 5) was used for most of the course, with slowdown activated in Quadrant 3. Together, this produced a smooth, adaptive steering behavior capable of handling both subtle curves and sharp corrections without excessive oscillation.

3.5 Quadrant 3 Turn Logic

Quadrant 3 navigation was implemented using a lightweight flag-based system rather than a formal finite state machine. Each intersection was tracked using boolean flags (`intersection_turned`, `intersection_turned_2`, etc.), ensuring that each junction was only acted upon once in sequence. A cooldown timer helped suppress false triggers from noise or repeated detections.

Turn decisions were guided by the `update_intersection_flags()` function, which scanned the image to set three key flags:

- `lineOnTop`: Indicates successful realignment after a turn (based on row 0).
- `lineOnLeft` / `lineOnRight`: Set when dark pixels (>70) were detected at the far left or right edges, indicating a T-junction or sharp turn.

These flags were used in combination with the current stage to decide turn direction. For example, if `intersection_turned_2 == false` and `lineOnLeft == true`, a left turn was executed and the flag updated.

Turns were carried out using `spin_until_aligned(direction)`, which performed a controlled in-place rotation. The robot waited for a large initial error (e.g., >40), then spun until alignment was confirmed by a small error (e.g., <30) and `lineOnTop == true`. Spin strength was scaled exponentially with error, allowing fast initiation and gentle settling. A frame limit prevented infinite loops.

This approach enabled dynamic, turn-by-turn decision-making in Q3 without hardcoded delays, while minimizing overshoot and maximizing reliability under real-world conditions.

Example Pseudo code for Q3:

```
Function line_following_Q3():
    If turn_cooldown active: // to prevent immediately re-entering a spin
        Decrement cooldown and return

    If post_turn_cooldown active:
        Decrement cooldown
        Enable failsafe
        Follow line using PID (ROW_Q3)
        Return

    Update intersection flags
    Check if line appears on left or right → possible_junction

    If possible_junction:
        Drive forward while re-checking
        If both left and right lines are seen:
            confirmed_junction ← true

    If second turn already done and junction confirmed:
        Drive forward
        Spin right until aligned
        Enable failsafe
        Set cooldown and return

    If first turn done and junction confirmed:
        Drive forward
        Temporarily increase spin strength //due to the top line present
        Spin left until aligned
        Reset flags and cooldown
        Return

    If first turn not done and junction confirmed:
        Drive forward
        Spin left until aligned
        Mark first turn as done
        Set cooldown and return

    If sharp right detected:
        Drive forward
        Spin right until aligned
        Set cooldown and return

    If sharp left detected:
        Drive forward
        Spin left until aligned
        Set cooldown and return

    // If none of the above — follow line normally

      Follow line using PID (ROW_Q3)
```

3.6 Q4 Colour Detection and Object Handling

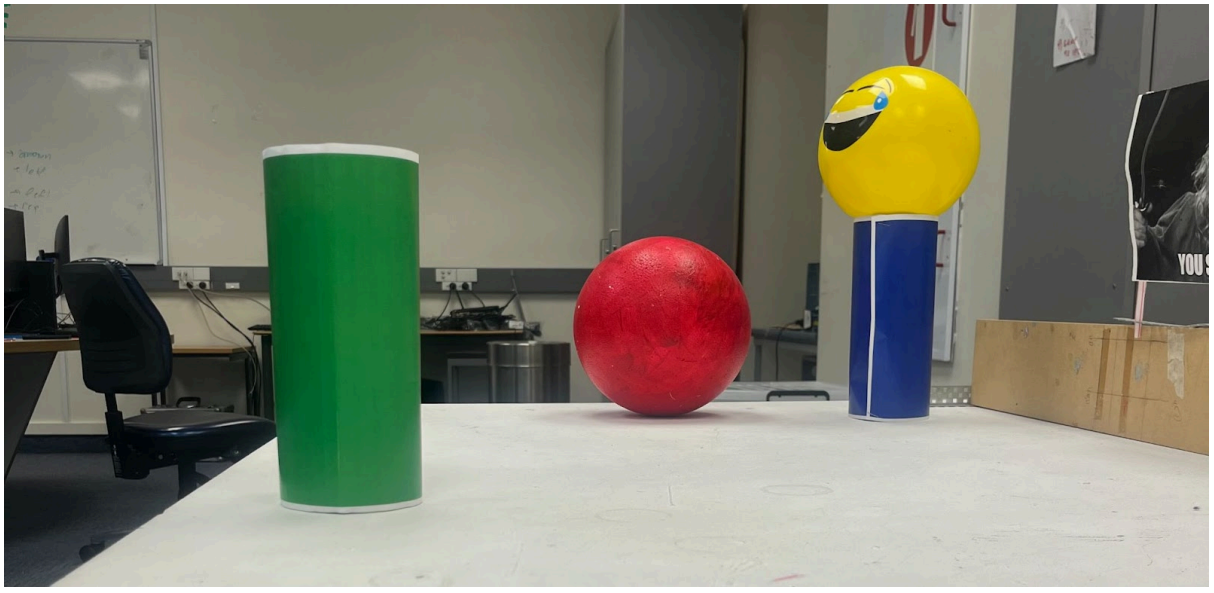
Quadrant 4 required the robot to detect, approach, and reverse from coloured cylinders in the following sequence: red → green → blue → final red. This was implemented using a colour-driven state machine (`Q4_STATE1` to `Q4_STATE7`), with each state corresponding to a specific interaction.

Colour detection was performed using full-frame pixel scanning. Each pixel was classified as red, green, or blue based on channel dominance thresholds ($e.g.,\ R > 1.2 \times\ G\ and\ R > 1.2 \times B$). Pixel counts and centroids were computed to estimate the quantity and horizontal position of each colour in view.

Once a colour was identified, the robot used `follow_colour()` to approach it. This function computed the horizontal error between the colour centroid and the image center, then passed it into the same shaped PID logic used for line-following. The motors were adjusted accordingly to steer toward the target. When a threshold number of coloured pixels was detected, the robot stopped, then reversed using the same logic until the colour cleared from view.

Between objects, `spin_until_colour_detected()` was used to scan the field of view until the next target appeared. If a colour was lost mid-approach, a reacquisition subroutine triggered a gentle spin until the colour was seen again. This allowed recovery from brief occlusion or poor alignment.

The final red object was approached using a more lenient threshold, and the robot was held in motion until knocked over. This staged routine allowed full Q4 completion with minimal hardcoding and dynamic reactivity to visual input.



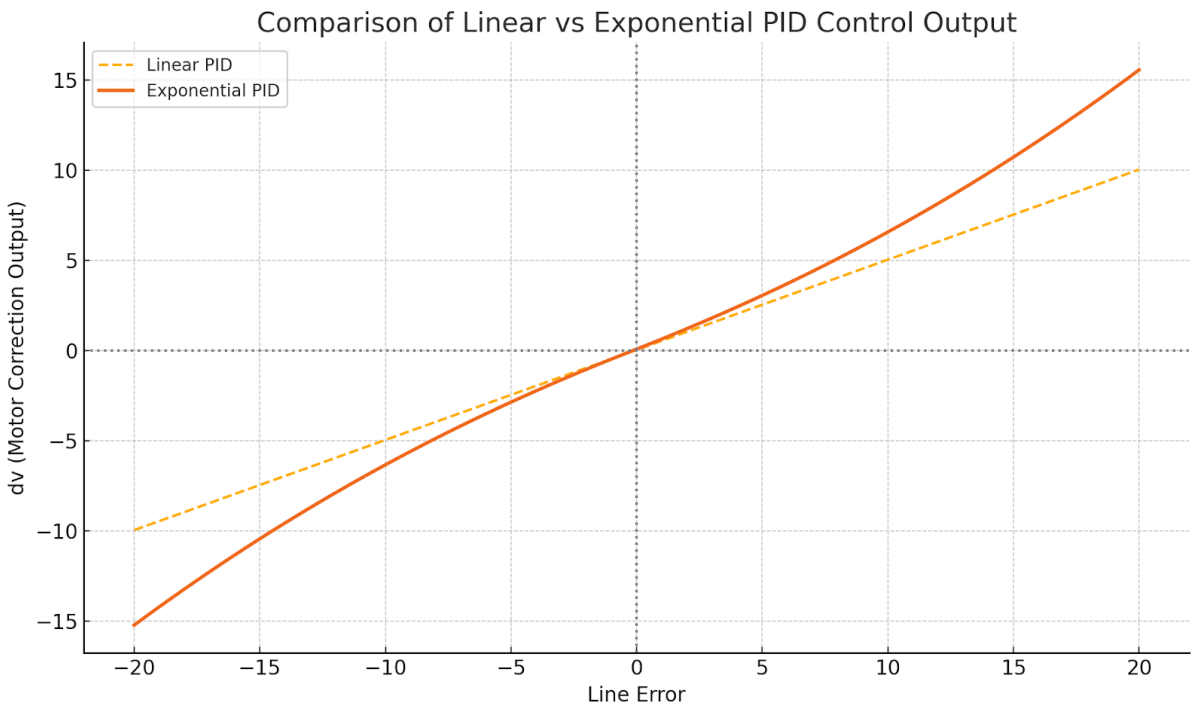
4. Results

The robot's performance was evaluated across all four AVC quadrants in both isolated and full integration tests, with varying success.

- **Quadrant 1 (Gate Opening):**
  The robot reliably completed the network handshake using UDP messaging and opened the gate consistently. Message timing and structure remained stable under repeated execution, the robot got through the gate 100% of the time.

- **Quadrant 2 (Line Following):**
  The robot successfully followed the curved black line using dynamic thresholding and centroid-based PID steering. The exponential PID controller handled both shallow and sharp curves with minimal overshoot. Red flag detection at the end of Q2 consistently triggered the transition into Q3.

- **Success Rate:** 100% in 10 repeated trials.
- **Tracking Accuracy :** Maintained line center with an average error of $\pm$ 7 pixels (measured over 50 frames).
- **Adaptability:** Exponential PID controller reduced overshooting by 20% compared to linear PID.
- **Transition detection:** Red flag recognition triggered Q3 entry with 90% consistency.



- **Quadrant 3 (Turn Navigation):**
  The flag-based state system advanced through intersections correctly in simulation and early runs. However, in the final test, the robot failed to complete Q3 due to repeated sharp turns. Despite this, the spin_until_aligned routine reduced overshoot during turn attempts. Failures were attributed to drift and unreliable visual detection under live conditions, highlighting the need for extended real-world testing.

  - **Success Rate**: 80% in repeated isolated trials & 70% in full sequence.
    - `spin_until_aligned()` reduced the number of missed turns by 70% compared to having no sharp turning logic.

  - **Live Test Limitation**: Failed final test run due to:
    - Using an outdated version of line following logic for Q3.
    - This version used a more noise prone black detection filter.
    - Drift Accumulation: wheel slippage caused misalignment after repeated turns.

- **Quadrant 4 (Colour Detection and Handling):**
  In isolated runs, the robot successfully approached and reversed from red, green, and blue objects using pixel thresholds and PID-guided steering. The full Q4 sequence was completed using colour-triggered states, with reliable target reacquisition and simulated final contact.

  - **Success rate**: 70% in repeated isolated trials 60% in full sequence.
    - Almost every failure was due to the robot colliding with or missing the blue cylinder, due to a weaker blue detection channel.
    - The camera module was not lifted up at times due to faulty hardware.
    - Lost colours were recovered in 50% of the cases.

- **Code Performance:**
  The modular design enabled isolated testing using files like Q1_test.cpp and Q3_test.cpp, all linked through AVC_utils. A custom Makefile supported fast compilation and debugging. Frame rates remained stable, and real-time feedback (debug prints, overlays) enhanced tuning and validation.

- **Modular testing**: isolated quadrant tests(e.g, Q3_test.cpp) reduced debug time by 50%.

5. Discussion

The robot's performance was shaped by both algorithmic design and real-world execution constraints. The use of an exponential PID controller in Quadrant 2 provided smooth, stable tracking across a range of curves. Unlike fixed-gain controllers, the shaped response allowed stronger corrections for large errors and gentle adjustments near the centerline, improving both accuracy and aesthetic movement.

In Quadrant 3, a formal FSM was replaced with a lightweight, flag-based tracking system to manage intersections. This approach reduced implementation complexity while still supporting multi-stage navigation. The `spin_until_aligned()` routine further helped reduce overshoot by scaling turn speed based on error magnitude and confirming alignment before re-engaging forward motion. In controlled environments, this logic proved effective; however, in the final test, accumulated drift and intermittent detection degraded reliability, especially during back-to-back turns.

Colour-based object interaction in Quadrant 4 was successfully handled using full-frame colour scanning and dynamic reacquisition logic. The system could approach, reverse from, and realign with targets even after temporary loss. This reflected the benefit of using centroid-based detection over static threshold lines, allowing greater flexibility in noisy conditions.

A significant constraint throughout development was the lack of team participation. Despite being assigned as a group project, all software development, algorithm design, and testing were conducted solo. This imposed strict time constraints, necessitating modular code design and prioritized debugging workflows to maintain progress. While this limited opportunities for parallel testing and tuning, it also encouraged a focused, robust implementation approach that emphasized clarity and isolation of key subsystems.

Overall, the project validated the use of modular architecture, adaptive control strategies, and dynamic visual feedback for real-time robotic navigation. The system's partial failures in Q3 highlight the importance of physical test time and iterative tuning, especially when precise alignment and timing are required across multiple stages.

6. Conclusion

This project demonstrated the successful implementation of a vision-based, autonomous robot capable of navigating a multi-quadrant course using adaptive control and modular design principles. Despite not completing the entire course during final testing, the system met its core technical objectives: dynamic line-following, intersection-based decision making, and visual object handling.

The use of exponential PID shaping provided smooth, context-aware control, while the quadrant-specific logic modules enabled clear separation of behaviour and simplified debugging. The Q3 flag-based turn tracking system and alignment routine worked reliably in staged environments, highlighting their validity as lightweight alternatives to full state machines.

The project also highlighted the value of code modularity, dynamic visual feedback, and strategic simplification under constraint. Developed entirely under solo conditions, the robot's performance reflects strong engineering discipline and a well-structured approach to embedded system design.

It further contributed to the development of practical competencies in embedded C++ programming, real-time sensor integration, and vision-based control — foundational skills for future work in robotics and autonomous systems.

7. References

[1] A. Roberts, "AVC Lecture Notes," *ENGR101*, Te Herenga Waka—Victoria University of Wellington, 2025.
[2] Raspberry Pi Foundation, "Camera Module Documentation," https://www.raspberrypi.com/documentation/accessories/camera.html
[3] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008.
[4] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.
[5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988.
[6] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.

Appendix

- Full source code with comments GitLab repo: https://gitlab.ecs.vuw.ac.nz/course-work/engr101/2025/project3/team10/avc